

CSC2062 – Assignment 1

Zohain Javed

40457802

3 March, 2026

Introduction

This report explores the development and statistical analysis of a feature extraction system designed to distinguish between "Living" and "Non-Living" hand-drawn objects. The dataset consists of 112 individual hand-drawn sketches including fruits (cherries, bananas, lemons), trees, and various inanimate objects (envelopes, golfclubs, pencils, wineglass). All sketches were drawn with a 1-pixel pencil on a 45x45 pixel canvas.

The primary goal of this study is to extract four specific geometric features: **pixel count (nr_pix)**, **height**, **width**, and **aspect_ratio**, and to determine through mathematical testing which of these characteristics most reliably and consistently determines whether an object is living or non-living. By using Python-based tools like `scipy.ndimage` to identify the drawings, this analysis determines whether simple computer vision metrics can effectively mimic human classification.

Section 1: Creating a dataset

In this section, I prepared a dataset consisting of 112 hand-drawn images. The dataset is balanced across eight categories: four living things (cherry, banana, lemon, tree) and four non-living things (envelope, golfclub, pencil, wineglass), with 14 images per category.

Data Creation Considerations:

- **Canvas and Tools:** Each image was drawn on a 45x45 pixel canvas in GIMP, using the pencil tool with a 1-pixel brush size.
- **Scaling and Design:** I ensured the drawings occupied the majority of the canvas area to maximize the reliability of feature extraction in later stages. The design was kept simple, representing the example drawings in the instructions document.
- **File Format:** Images were initially exported in PGM (ASCII) format to allow for easy conversion via Python.

Thresholding and Conversion:

To prepare the data for statistical analysis, I wrote a Python script (`section1_code.ipynb`) to automate the conversion of the PGM file into a CSV binary matrix.

- **Logic:** The script reads the greyscale pixel values (0–255) and applies a threshold of 128.
- **Mapping:** Following the assignment specifications, pixel values less than 128 were mapped to **0 (black/object)**, and values 128 or greater were mapped to **1 (white/background)**.
- **Output:** The resulting 45x45 binary matrices consisting of 0s and 1s were saved as comma-delimited CSV files using the required naming convention:
`40457802_[label]_[index].csv`.

All the PGM and CSV files were saved in the “**images**” folder.

Data Verification

To ensure the conversion from PGM to CSV was successful, I opened one of the CSV files in a text editor (notepad). The sample image (a pencil) on the left, as shown in Figure 1, is successfully represented by the binary matrix containing 0s and 1s on the right.

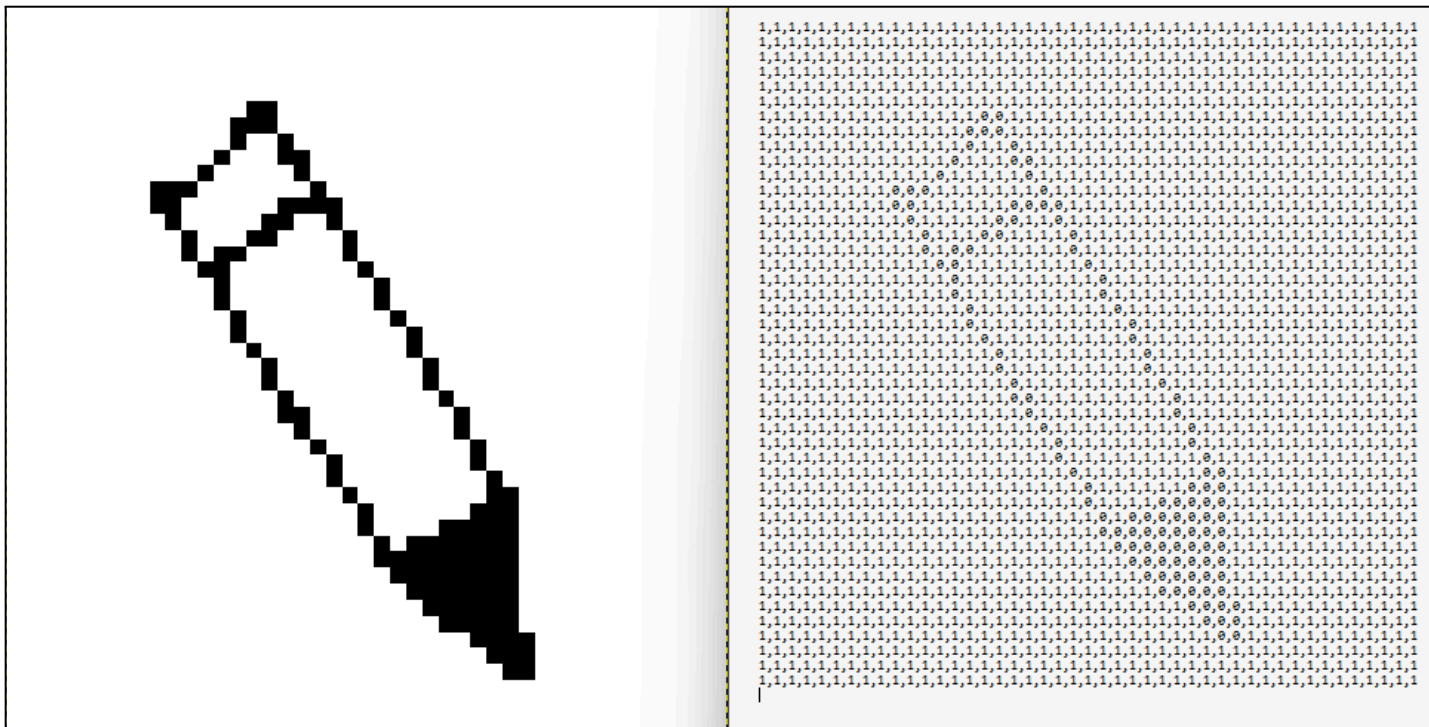


Figure 1: Visualization of the 45x45 binary matrix for a pencil drawing. (pencil_14.pgm)

Section 2: Feature Engineering

In this section, I extracted 16 distinct numerical features from the raw 45x45 matrix data. The goal was to quantify the spatial, geometric, and topological characteristics of each hand-drawn image to create a dataset viable for statistical analysis and machine learning classification.

The feature extraction was implemented in Python ([section2_code.ipynb](#)). I utilized `numpy` for efficient matrix operations, and `scipy.ndimage.label(clabel)` for labeling connected components. The data was finally exported to a CSV table format ([40457802_features.csv](#)).

1. Pixel Distribution and Density Metrics (Features 1–7)

These features measure the quantity and distribution of black pixels:

- **Feature 1 (nr_pix):** The total count of black pixels. I calculated this by using `np.where(matrix == 0)` to get the coordinates of all black pixels and taking the `len()` of the resulting array.

- **Features 2–5:** The number of rows and columns containing exactly 1 or 2 black pixels (`rows_with_1`, `cols_with_1`, `rows_with_2`, `cols_with_2`). I first calculated the number of black pixels in every row and column using `np.sum` along `axis=1` and `axis=0`. I then used `np.sum` again on the resulting list to count how many of those rows/cols had exactly 1 or 2 pixels.
- **Features 6–7:** The number of rows and columns containing 3 or more black pixels (`rows_with_3p`, `cols_with_3p`).

2. Geometric and Bounding Box Metrics (Features 8–12)

These features define the physical dimensions and proportions of the drawing:

- **Features 8–9 (`height`, `width`):** Using the coordinates found from `np.where`, I subtracted the minimum row/column index from the maximum row/column index to find the vertical and horizontal span.
- **Feature 10 (`aspect_ratio`):** Calculated as `width / height`.
- **Features 11–12 (`maxrow`, `maxcol`):** The maximum number of black pixels found in any single row or column, found by applying `np.max()` to the row and column counts.

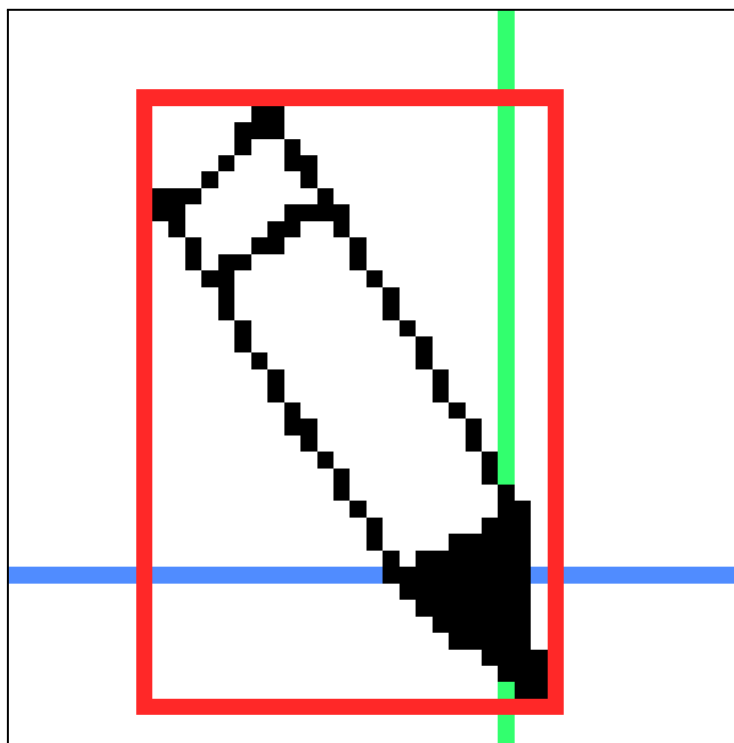


Figure 2: Visual verification of geometric extraction. The red box encloses the figure, illustrating the dynamic calculation of `height` and `width` relative to the 45x45 canvas. The blue horizontal and green vertical lines indicate the identified rows and columns of maximum pixel density (`maxrow` and `maxcol`). (`pencil_14.pgm`)

3. Topological Connectivity (Features 13–15)

These features describe the structural complexity and "hollowness" of the shapes:

- **Feature 13 (`connected_areas`):** The number of separate regions of connected black pixels. I passed `matrix == 0` into the `clabel` function along with a 3x3 array of ones to enforce 8-connectivity, since the center element of the 3x3 matrix has 1s in all 8 directions (neighbours).

- **Feature 14 (eyes):** The number of separate whitespace regions completely surrounded by a ring of connected black pixels. I labeled all white pixels (`matrix == 1`) using a cross-shaped 4-connectivity matrix as the `struct` input in the `clabel` function. To avoid counting the background canvas as an "eye," I used array slicing (e.g., `labeled_white[0, :]`) to find any white regions touching the very edge of the canvas, and added these regions to a `set`. This set of regions was then subtracted from the total count.
- **Feature 15 (hollowness):** Hollowness was calculated as the ratio of total white pixels across all eye regions divided by `nr_pix`. To calculate the white pixels in eye regions, I created a boolean mask `is_eye_pixel` that selected white pixels while excluding any labels found in the background set (created in Feature 14) using `~np.isin`.

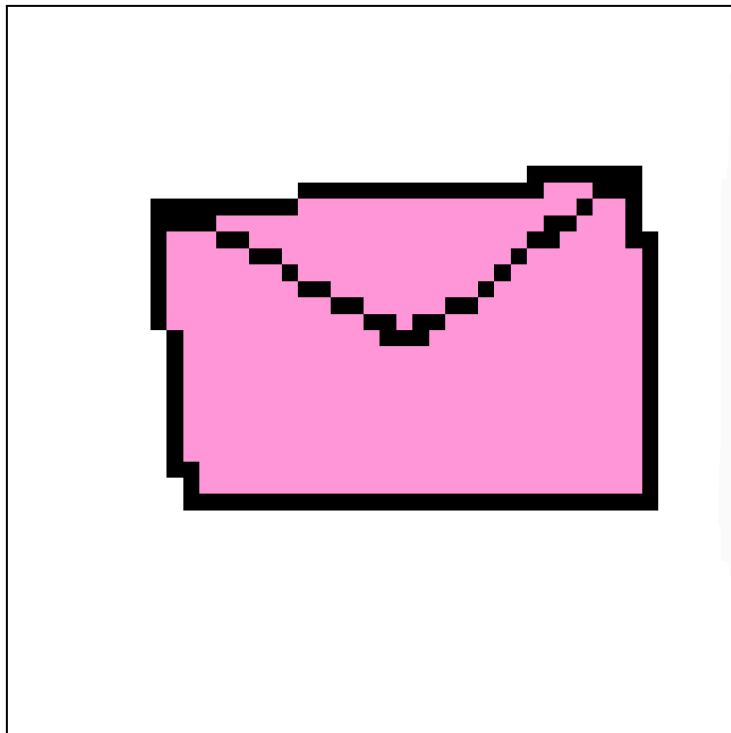


Figure 3: Topological analysis of enclosed space. The pink colored region represents the internally connected white pixels ("eyes") isolated via a 4-connectivity flood-fill algorithm (`scipy.ndimage.label`). The envelope has two distinct eye regions. (`envelope_13.pgm`)

4. Custom Feature: Roughness (Feature 16)

- **Feature 16 (custom):** A custom metric that calculates how "rough" or messy a drawing is by finding the ratio of stray, dead-end pixels to the total number of pixels.

For my custom feature, I wanted to identify "stray" pixels to quantify how messy or complex a drawing was. My initial idea was actually much more complex than my final code. I originally tried to classify pixels based on their exact combination of neighbors, and manually code multiple cases of specific 3x3 geometric patterns to catch things like "flat line bumps," situations where a single pixel sits on top of an otherwise straight line. I realized implementing this would require a much more complex system, as I will discuss later.

My initial ruleset for identifying a "stray" pixel was:

- Pixels with 0 orthogonal and 0 diagonal neighbour.

- Pixels with 0 orthogonal and 1 diagonal neighbour.
- Pixels with 1 orthogonal and 1 diagonal neighbour, where the diagonal neighbour is not considered “stray” (bump on the edge of a straight line).
- Pixels with 1 orthogonal and 2 diagonal neighbours, where the diagonal neighbours are not considered “stray” (bump on the middle of a straight line).

I ran into major difficulties trying to code this approach. First, manually checking the four rotational cases for these patterns was extremely tedious. More importantly, I couldn't simply just check for cases like `ortho=1` and `diag=2`, as that didn't just catch the bumps I wanted; it also kept falsely flagging legitimate parts of the drawing, like thick corners or intersections, treating them as stray pixels. Fixing this would require the complex multi-pass system I alluded to earlier, just to check whether a neighbour was also “stray” or not before I could safely count the target pixel.

My second attempt, using much simpler constraints, checked if `(ortho == 0) and (diag <= 1)`. The issue with this ruleset was that the feature value for 95% of the dataset came out to be essentially 0. I realized this constraint was far too strict for determining if a pixel is considered stray, meaning the feature wouldn't actually help differentiate between the classes. I had to broaden the constraint.

In my final code, I just loop through the coordinates of every black pixel using `zip(rows, cols)` and count its neighbors. If the combined total of a pixel's orthogonal and diagonal neighbors is 1 or 0 (`ortho + diag <= 1`), it gets flagged as a `stray_pixel`. This single mathematical condition completely bypassed the need to check rotations or neighbor-states, while successfully ignoring thick corners and perfectly catching the true dead-ends. The final feature is calculated as the ratio of `stray_pixels / nr_pix`.

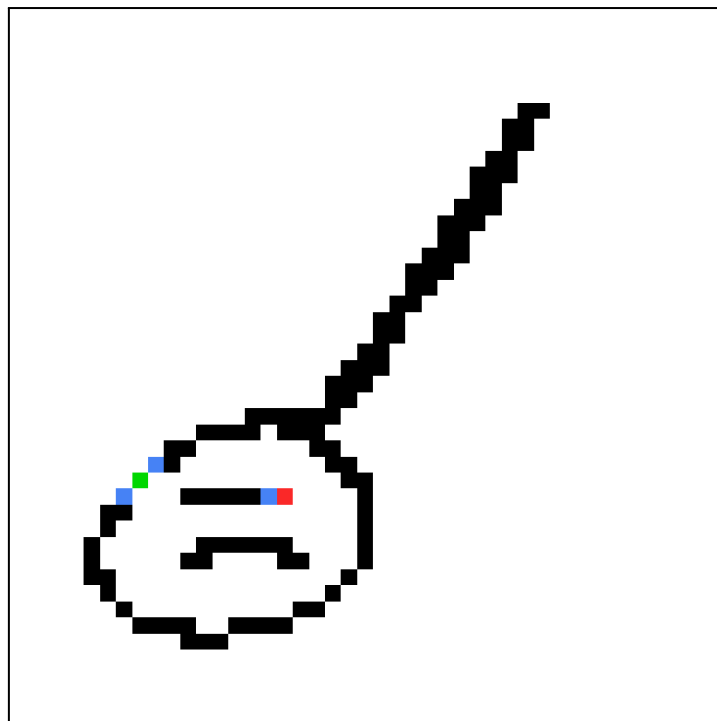


Figure 3: Neighbour classification logic. The **red** pixel represents a “Stray” pixel, as it is a terminal endpoint (`ortho + diag = 1 + 0`), contributing to the “Roughness” feature. The **green** pixel is correctly identified as a “Safe” connector due to its diagonal connectivity (`ortho + diag = 0 + 2`), demonstrating how the algorithm preserves topological continuity. (golfclub_08.pgm)

Section 3: Statistical analysis

In this section, I perform a statistical evaluation of the four geometric features: `nr_pix`, `height`, `width`, and `aspect_ratio`, in order to determine how they are distributed across the dataset. The goal is to identify which features show the most significant differences between the "Living" and "Non-Living" classes. By using a combination of visual histograms, normality tests, and T-tests, I can determine which characteristics are most useful for reliably distinguishing between the two categories.

Section 3.1: Histogram Analysis and Patterns

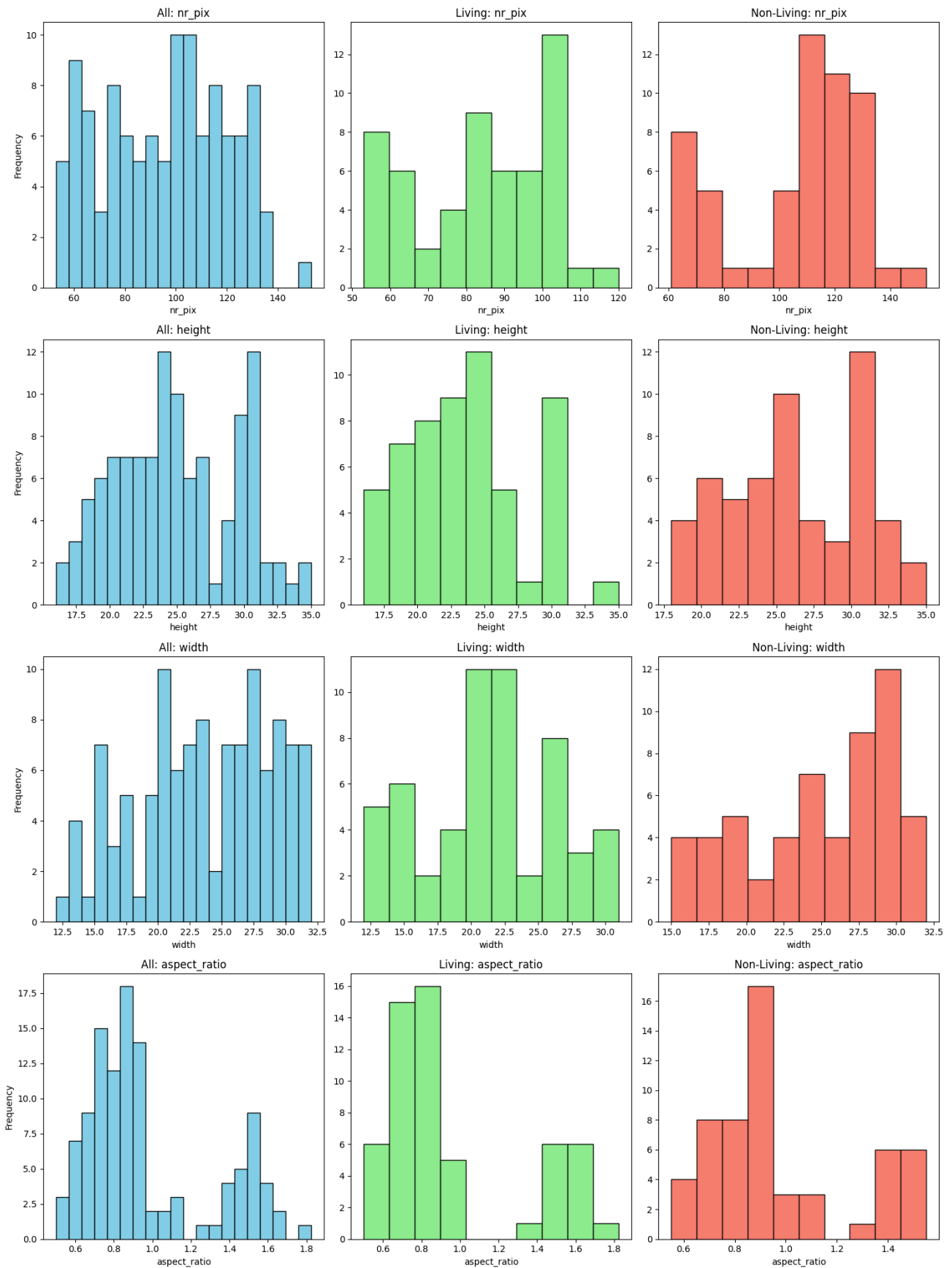


Figure 5: Histograms grid for geometric and density features across the full dataset ($n=112$), Living things ($n=56$), and Non-Living things ($n=56$).

To evaluate the characteristics of the dataset, I generated histograms covering the four main geometric features: `nr_pix`, `height`, `width`, and `aspect_ratio`. Each feature is plotted across

three subsets of the dataset: the full dataset (n=112), living things (n=56), and non-living things (n=56), giving a total of twelve histograms.

I arranged the histograms in a 4x3 grid using `matplotlib.pyplot.subplots`, with one row per feature and one column per dataset subset. This layout allows direct visual comparison of the distributions between the two classes.

For the `nr_pix` feature, there is a clear difference between living and non-living things. The living objects are spread across a lower range, with most values appearing roughly between 55 and 105. But the shape is uneven, with visible gaps rather than a smooth curve. In contrast, the non-living objects are slightly left skewed, shifting towards higher values typically between 100 and 135. There is a noticeable gap between the values 80-100. The combined dataset shows a wider, more irregular spread due to the overlap of both these groups.

For the `height` feature, the living subset shows slight right skewness, with most values spread uniformly in the lower range. There is a peak around the value 30. The non-living subset is relatively more symmetric and bimodal, with less skewness. All three subsets show two clear peaks, while the living subset being the least symmetric of the three. Larger heights appear to be more common in non-living objects.

For the `width` feature, the living subset shows a slight trimodal distribution, while being the most symmetric of the three. The non-living subset is noticeably left-skewed, showing a strong distinction from the living subset. The combined dataset shows two major clusters, with slight left skewness.

For the `aspect_ratio` feature, the distributions for the living and non-living items overlap significantly. Both subsets contain values mostly in the range of approximately 0.6 to 1.6. There seems to be a slight positive skew, especially for the living objects. There are very few values in the 1-1.4 range, resulting in bimodal distribution with slight right skew. The full dataset follows the same trend, with two clearer clusters and relatively more positive skewness.

Overall, none of the features appear to follow significantly normal distribution based on visual inspection. The `height` and `aspect_ratio` features show the most noticeable skew. Non-living distribution seems to be more skewed than living distribution across all features. The `width` feature shows the most distinct separation between living and non-living objects, with the living subset showing a peak in the middle while the non-living subset is noticeably left skewed. In contrast, `aspect_ratio` shows substantial overlap between the two subsets and is therefore less useful for distinguishing between the two based on this feature alone.

Section 3.2: Normal Distribution Fit

1. Results: Based on the calculations for the `nr_pix` feature across the entire dataset (n=112):

- **Mean (μ):** 95.63
- **Variance (σ^2):** 575.54

2. Visual Comparison: I plotted a theoretical Normal distribution curve (the red line) over the actual histogram of the `nr_pix` data. By comparing the two, It can clearly be observed that the Normal distribution does not provide a perfect fit for this dataset.

3. Validity of Assumptions: The assumption that the `nr_pix` data follows a Normal distribution is **weak**. While the bell curve captures the general range of the data, the actual data appears **broadly and irregularly distributed** across its range. This suggests that the drawings vary significantly in

their pixel count, and a single mathematical model (like the Normal distribution) cannot accurately be used to describe the diversity of the hand-drawn items in this dataset.

- Reasoning:** The Normal distribution fails to accurately represent the `nr_pix` feature because the actual data is irregularly distributed rather than clustering around a single central peak, which can be seen in the Normal distribution. While a Normal distribution is perfectly symmetrical and "bell-shaped," the hand-drawn dataset results in significant variance that the theoretical model cannot capture. This suggests that the assumption of normal distribution for the combined dataset is not valid.

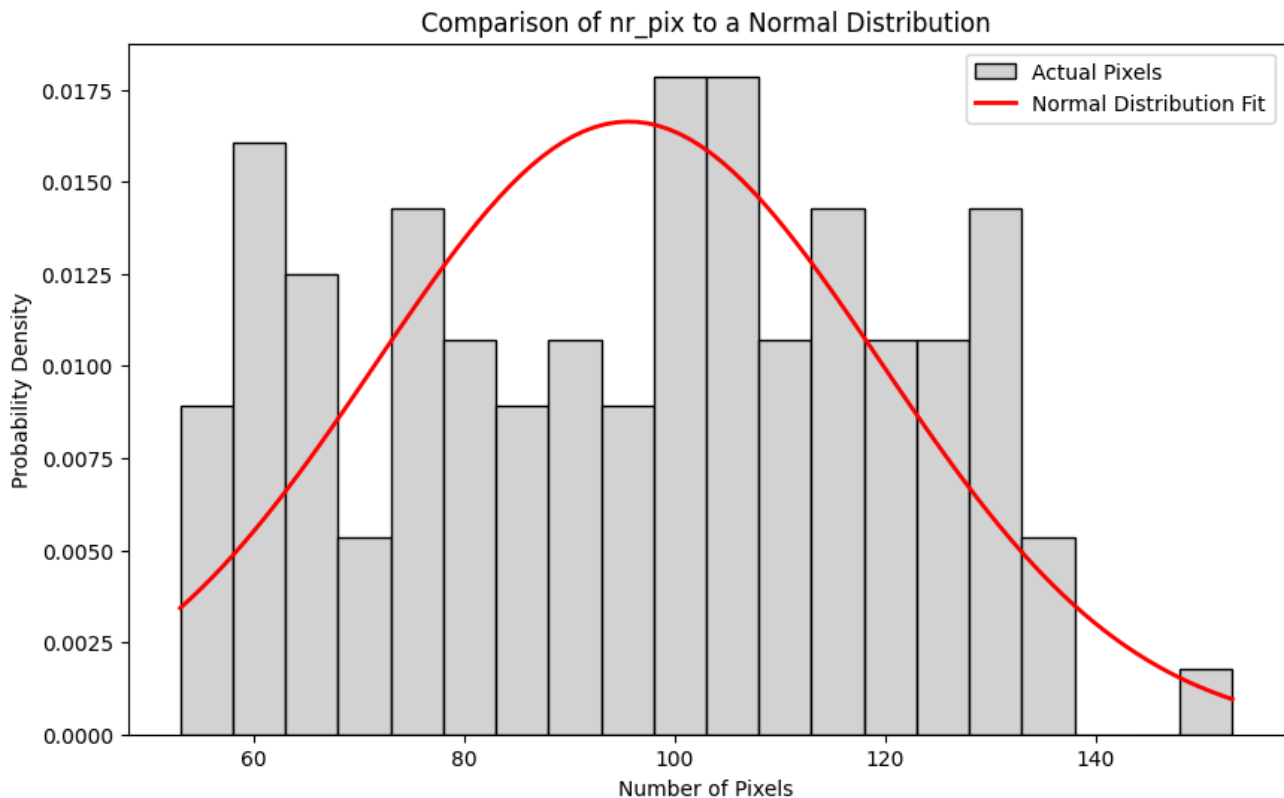


Figure 6: Histogram of `nr_pix` (Full Dataset, $n=112$) with Normal Distribution Fit.

Section 3.3: Probability Threshold

1. Theoretic Calculation: Assuming a normal distribution based on the previously calculated mean ($\mu = 95.63$) and variance ($\sigma^2 = 575.54$), the `nr_pix` value such that a randomly sampled image would have a 5% probability of having a greater value was calculated using the Percent Point Function (`scipy.stats.norm.ppf`):

- Calculated 5% Cut-off Value: 135.09**

2. Visual and Data Validation: This value represents the theoretical "upper tail" where only the top 5% of observations should reside. In a perfectly normal distribution of 112 items, we would expect approximately **5.6 items** (5% of 112) to exceed this threshold.

However, after inspecting the actual frequency counts for `nr_pix`, I observed that only **4 out of 112** observations (approx. **3.5%**) exceeded the 133.00 mark. So, less than 3.5% of the observations exceeded the pixel count beyond the 135.09 threshold.

3. Conclusion: The discrepancy between the theoretical 5% threshold (135.09) and the actual lack of data points in that range further confirms the assessment in Section 3.2. The `nr_pix` distribution has a "thinner" upper tail than a true Normal distribution, proving that the normal model slightly overestimates the frequency of high-pixel-count drawings in this specific dataset, which can be observed in Figure 6.

Section 3.4: Hypothesis Testing (T-tests)

1. Objective: To numerically determine which features most effectively distinguish between Living and Non-Living objects, I performed independent samples T-tests (`scipy.stats.ttest_ind`). This statistical method evaluates whether the difference between the means of the two groups (Living vs. Non-Living) is significant or simply due to random chance. I used a standard significance threshold of 5% ($\alpha = 0.05$).

2. Results Table: The T-test results for each of the four geometric characteristics were as follows:

Feature	P-Value	Statistical Significance ($p < 0.05$)
<code>nr_pix</code>	0.0000001552	Highly Significant
<code>height</code>	0.0026816742	Significant
<code>width</code>	0.0004534143	Significant
<code>aspect_ratio</code>	0.7065059180	Not Significant

3. Analysis and Comparison: The T-test results provide a better understanding than the initial visual inspection in Section 3.1. While I previously identified `width` as the most distinct separator, the statistical results show that `nr_pix` is actually the most mathematically robust discriminator, yielding the lowest P-value ($p \approx 1.55 \times 10^{-7}$).

However, my initial observation regarding `width` was still largely accurate, as it remains the **second-best indicator** for classification with a really significant P-value ($p \approx 4.53 \times 10^{-4}$). `height` follows as the third-best feature ($p \approx 2.68 \times 10^{-3}$).

Most importantly, the statistical test confirmed my initial visual assessment that `aspect_ratio` is the least useful feature for making the distinction between the two groups. With a P-value of **0.7065**, it is the only characteristic that fails to meet the 0.05 significance threshold, proving there is no meaningful difference in the aspect ratios of living versus non-living objects in this specific sample.

Section 3.5: Correlation and Association

1. Results: I calculated the Pearson correlation coefficients to identify linear associations between the geometric features. The resulting correlation matrix is as follows:

	nr_pix	height	width	aspect_ratio
nr_pix	1.00	0.25	0.86	0.50
height	0.25	1.00	0.00	-0.63
width	0.86	0.00	1.00	0.76
aspect_ratio	0.50	-0.63	0.76	1.00

2. Analysis: The strongest linear association was found between **nr_pix** and **width**, with a high positive correlation coefficient of **0.86**. This is logically consistent: as a hand-drawn object becomes wider, it covers more area on the canvas, leading to a higher total pixel count.

Conversely, the weakest correlation was observed between **height** and **width** (approx. **0.00**). This suggests that in this specific dataset, the vertical and horizontal dimensions of the drawings vary entirely independently of one another.

Additionally, **height** shows a moderate negative correlation with **aspect_ratio** (**-0.63**), which makes sense mathematically as height is inversely related to the aspect ratio ($\text{aspect_ratio} = \text{width} / \text{height}$).

Section 4: Regression and Machine Learning

Section 4.1: Predicting Aspect Ratio (Linear Regression)

Instead of just calculating **aspect_ratio** like in Section 2, I tried to predict it using other features. I chose **Linear Regression** for this because **aspect_ratio** is a continuous number. To keep the model simple, I only used **width** and **height** as my predictors. I didn't include **nr_pix** because it doesn't actually define the shape's proportions, and using just the two dimensions (W and H) is the most direct way to get the ratio without overcomplicating the model.

Section 4.2: Regression results

I used **Mean Squared Error (MSE)** to see how far off my predictions were.

- **MSE:** 0.021
- **R-Squared:** 0.94

The high **R²** value shows that width and height together explain almost all the changes in the aspect ratio, which makes sense since the ratio is literally just one divided by the other.

Section 4.3: Logistic Regression for Classification

Following the results from Section 3.4, I used **nr_pix** as the main feature to fit a **Logistic Regression** model, since it was my most significant "Living vs. Non-Living" divider.

The fitted curve (the Sigmoid function) shows a clear "S" shape. It stays near 0 (Living) for lower pixel counts and then jumps toward 1 (Non-Living) once the pixel count crosses the 95-100 range. This curve acts as the decision boundary for the computer to sort the sketches.

Section 4.4: Interpretation

The results show that while Linear Regression is great for predicting a specific value like a ratio, **Logistic Regression** is much better for the "Yes/No" task of classifying objects. The fact that my **nr_pix** P-value was so low (0.0000001552) means the Logistic model has a very easy time drawing a line between the two groups.

Conclusions

Overall, this project shows that the best way to tell my living drawings apart from the non-living ones is by looking at **pixel count (nr_pix)** and **width**. My T-tests proved these were the most reliable features, while **aspect_ratio** was basically useless for making this distinction, since the P-value was so high (**0.70**). Even though the 45x45 canvas made everything a similar size and messed with the true scale of the objects, the math still found a clear difference in most categories. If I did this again, I would use a bigger variety of drawings and maybe more complex shapes, but for this dataset, the pixel density was the clear winner.

References

1. **Bruce, P., & Bruce, A. (2017).** *Practical Statistics for Data Scientists*. O'Reilly Media. (For T-test and Correlation theory).
2. **SciPy Development Team (2026).** *scipy.ndimage.label Documentation*. [Online] Available at: <https://docs.scipy.org/> (Used for identifying and labeling connected components in the image).
3. **scikit-image Development Team (2026).** *skimage.measure.label Documentation*. [Online] Available at: <https://scikit-image.org/> (Research on region labeling and feature extraction).
4. **Matplotlib Development Team (2026).** *Matplotlib User Guide: Visualization*. [Online] Available at: <https://matplotlib.org/>
5. **Pandas Development Team (2026).** *Pandas Documentation*. [Online] Available at: <https://pandas.pydata.org/docs/>
6. **Generative AI Tools (Gemini/ChatGPT).** Used for Python code debugging, clarifying statistical concepts, and title brainstorming.
7. **Grammarly.** Used for final spelling and grammar verification.